

CS3807: Deep Learning Laboratory

Experiment 4: Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

B.Tech Artificial Intelligence & Data Science, Semester V

AY 2026–27

1 Objective

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand and implement transfer learning.
- Fine-tune a pretrained CNN model and evaluate its performance.

2 Introduction

CNNs became the standard for computer vision because they learn hierarchical features directly from image data instead of relying on hand-crafted descriptors. Starting from LeNet-5, the field moved through AlexNet, VGGNet, GoogleNet and ResNet, with each generation solving a specific bottleneck of the previous one: underfitting, overfitting, filter efficiency, and vanishing gradients respectively.

Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical CNN
AlexNet	2012	ReLU activation, Dropout, GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale features
ResNet	2015	Residual learning via skip connections

3 Architecture Overview

LeNet-5 is a 7-layer network with two convolution and average-pooling stages, originally built for handwritten digit recognition; small, fast, but too shallow for natural images.

AlexNet scaled this up to 8 layers and 61M parameters, and its use of ReLU, Dropout and GPU training is what made deep networks practical in the first place.

VGG16 showed that stacking small 3×3 filters (16 layers, 138M parameters) gives the same receptive field as larger filters while adding more non-linearity.

GoogleNet replaced single-size filters with an Inception module that runs 1×1 , 3×3 , 5×5 convolutions and pooling in parallel and concatenates them, using 1×1 convolutions to cut computation (22 layers but only 6.8M parameters).

ResNet tackled the vanishing-gradient problem in very deep networks using skip connections: instead of learning $H(x)$ directly, a residual block learns $F(x) = H(x) - x$, so the output is $F(x) + x$. This is what allows networks like ResNet50 (50 layers, 25.6M params) to train reliably.

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

4 Transfer Learning

Instead of training a CNN from scratch, transfer learning reuses a model already trained on ImageNet: the pretrained convolutional base is frozen, a new classifier head is attached on top, trained on the target dataset, and then the top few convolution blocks are unfrozen and fine-tuned at a low learning rate. This converges faster and needs far less data than training from scratch, since the frozen base already knows generic edges, textures and shapes.

For this experiment, **MobileNetV2** was used as the pretrained base (ImageNet weights) because it gives a good accuracy-to-compute trade-off for CIFAR-10 sized experiments on limited GPU time.

5 Dataset

CIFAR-10: 50,000 training and 10,000 testing images, 10 balanced classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), each image $32 \times 32 \times 3$. The dataset was loaded locally from Google Drive (mounted via `drive.mount()`) rather than downloaded at runtime, and pixel values were normalized to $[0, 1]$.

Train: (50000, 32, 32, 3) (50000, 10)
 Test : (10000, 32, 32, 3) (10000, 10)

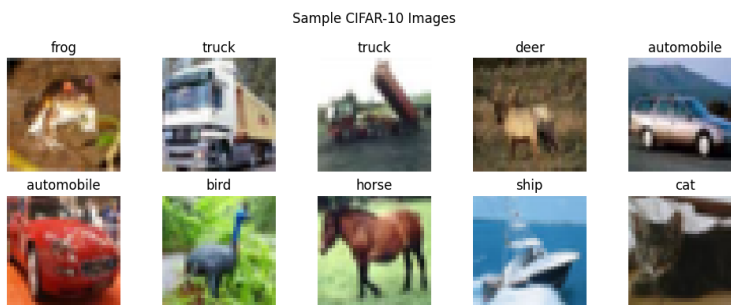


Figure 1: Sample images from the CIFAR-10 training set.

Inference: The images vary a lot in background and appearance, which can make classification harder and makes augmentation useful.

6 Model Architecture

The pretrained MobileNetV2 base (frozen) is preceded by a resizing layer ($32 \times 32 \rightarrow 96 \times 96$, since MobileNetV2 expects larger inputs) and followed by Global Average Pooling, a 256-unit Dense (ReLU) layer, Dropout(0.3), and a 10-way Softmax output.

Model: Functional		
Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 32, 32, 3)	0
resizing (Resizing)	(None, 96, 96, 3)	0
multiply (Multiply)	(None, 96, 96, 3)	0
true_divide (TrueDivide)	(None, 96, 96, 3)	0
subtract (Subtract)	(None, 96, 96, 3)	0
mobilenetv2_1.00_96 (Functional)	(None, 3, 3, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 256)	327,936
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 10)	2,570
Total params: 2,588,498 (9.87 MB)		
Trainable params: 330,596 (1.26 MB)		
Non-trainable params: 2,257,904 (8.61 MB)		

Figure 2: Model summary: 2.59M total parameters, of which only 330K are trainable in the frozen phase.

7 Training

Phase 1 (frozen base): Adam, $\text{lr} = 0.001$, batch size 32, 10 epochs, categorical cross-entropy.

Phase 2 (fine-tuning): last block of MobileNetV2 unfrozen, Adam, $\text{lr} = 10^{-5}$, 8 more epochs.

Before fine-tuning val acc : 0.876

After fine-tuning val acc : 0.883

8 Mandatory Plots

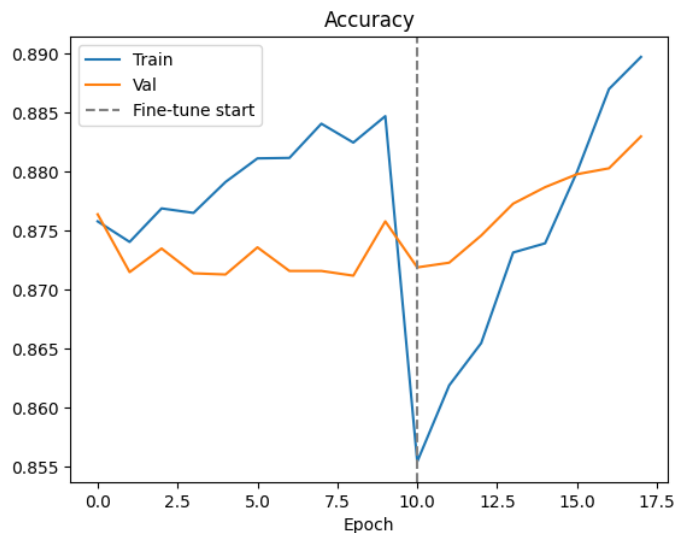


Figure 3: Training vs Validation Accuracy.

Inference: Validation accuracy stays around 87.5% before fine-tuning, then improves to about 88.3% after fine-tuning, showing that unfreezing the deeper layers helped the model.

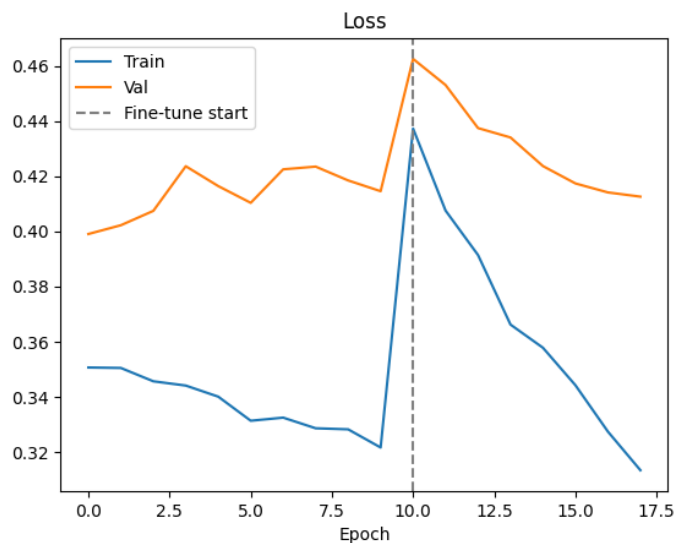


Figure 4: Training vs Validation Loss.

Inference: Loss spikes when fine-tuning starts, then steadily decreases. The gap between training and validation loss remains, suggesting some overfitting but overall improvement.

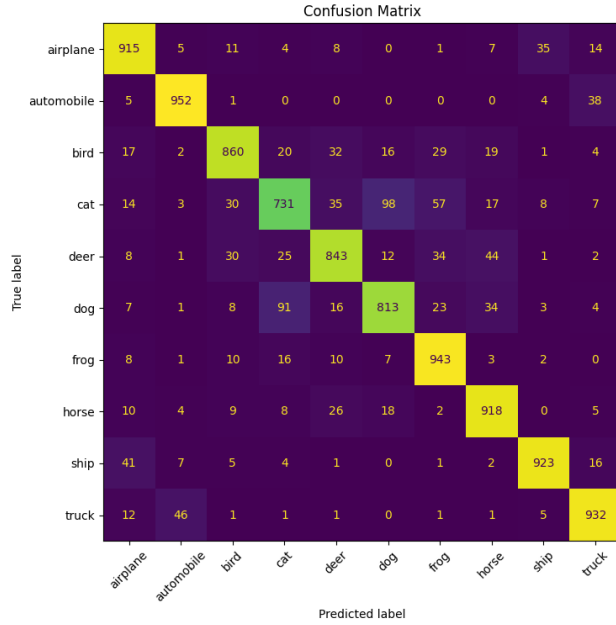


Figure 5: Confusion Matrix on the test set.

Inference: Cat is the hardest class to classify, with many confused with dog and frog. Automobile and truck are also often confused, while frog is classified particularly well.

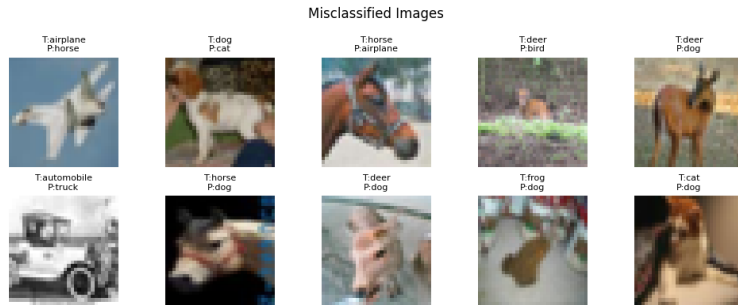


Figure 6: Sample misclassified images.

Inference: Most errors occur between visually similar classes, such as horse and bird or deer and dog. Some low-quality images are also harder to classify correctly.

9 Results

9.1 Performance Metrics

Metric	Value
Training Accuracy	0.890
Testing Accuracy	0.883
Precision	0.882
Recall	0.883
F1-score	0.882
Training Time	N/A
Total Parameters	2,588,490

9.2 Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.88	0.92	0.90	1000
automobile	0.93	0.95	0.94	1000
bird	0.89	0.86	0.88	1000
cat	0.81	0.73	0.77	1000
deer	0.87	0.84	0.85	1000
dog	0.84	0.81	0.83	1000
frog	0.86	0.94	0.90	1000
horse	0.88	0.92	0.90	1000
ship	0.94	0.92	0.93	1000
truck	0.91	0.93	0.92	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000

9.3 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Remarks
LeNet-5	60K	~65	Too shallow for natural images
AlexNet	61M	~75	First deep breakthrough
VGG16	138M	~85	Strong but heavy
GoogleNet	6.8M	~88	Efficient, multi-scale
MobileNetV2 (this run)	2.59M	88.3	Transfer learning, lightweight

10 Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

Before AlexNet, deep networks were difficult to train due to vanishing gradients. AlexNet used ReLU, Dropout and GPU training, allowing a much deeper CNN to be trained effectively.

2. Why does VGG16 use only 3×3 convolution filters?

Two 3×3 convolutions give a similar receptive field to one 5×5 convolution, but use fewer parameters and add an extra non-linearity.

3. Explain the advantages of the Inception module.

Instead of picking one filter size and hoping it's right, the Inception module runs 1×1 , 3×3 , 5×5 convolutions and a pooling branch in parallel on the same input and concatenates the outputs, so the network captures features at multiple scales at once. The 1×1 convolutions placed before the bigger filters act as a bottleneck that reduces the channel depth first, which is why GoogleNet manages 22 layers with only 6.8M parameters, far fewer than VGG16's 138M for comparable accuracy.

4. What is the purpose of residual learning?

In very deep plain networks, gradients shrink as they're backpropagated through many layers, so early layers barely update; this is the vanishing gradient problem. A residual block reformulates the target mapping as $F(x) = H(x) - x$ and adds the input back, $\text{Output} = F(x) + x$, so gradients have a direct path through the skip connection even if F itself is hard to learn. This is what lets ResNet go to 50+ layers and still train well.

5. Differentiate LeNet and ResNet.

LeNet is a 7-layer network with $\sim 60\text{K}$ parameters, average pooling, and no skip connections, built for small grayscale digit images and trains in seconds even on a CPU. ResNet (50 layers, 25.6M params for ResNet50) uses residual/skip connections specifically to allow this much greater depth to train without vanishing gradients, and is meant for large, complex, real-world image datasets. Basically the same core idea ($\text{conv} \rightarrow \text{pool} \rightarrow \text{FC}$) scaled up with one key architectural fix.

6. What is Transfer Learning?

Taking a model already trained on a large dataset (usually ImageNet) and reusing its learned weights for a new, related task instead of starting from random initialization, typically by freezing the convolutional base, replacing the classifier head, and training only the new head on the target dataset.

7. Why is fine tuning required?

Training only the new head with the base frozen gets you decent accuracy quickly, but the frozen features were learned for ImageNet's 1000 classes, not specifically for CIFAR-10. Unfreezing the last few convolution blocks and training them at a very low learning rate lets those layers adjust slightly to the new dataset's specific patterns, which is exactly what pushed our validation accuracy from 0.876 to 0.883.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated (atrous) convolution inserts gaps between kernel elements so the filter covers a larger receptive field without adding parameters or changing the output size; useful in segmentation where you want context without downsampling. Transpose convolution does the opposite job: it upsamples a feature map to a larger spatial size using learnable weights, which is how decoders in autoencoders/GANs/segmentation networks reconstruct full-resolution output from a compressed representation.

9. Why do pretrained models converge faster?

Pretrained models already have useful features such as edges, textures and shapes. Starting from these learned features is much faster than starting from random weights.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has about 60K parameters and can be trained on a CPU in a few minutes for MNIST-sized data, computationally almost trivial by modern standards. ResNet50 has roughly 25.6M parameters and 50 layers, needing GPU acceleration and considerably more FLOPs per forward/backward pass. The trade-off is accuracy and capacity: ResNet can represent far more complex functions and handle harder datasets, but at a computational cost that's orders of magnitude higher than LeNet.

11 Additional Exercises

1. **VGG16/ResNet50 instead of MobileNetV2:** swap the base model line to `tf.keras.applications.VGG16` or `ResNet50(...)` with the matching `preprocess_input`; the rest of the training pipeline stays the same.
2. **Adam vs SGD:** re-compile with `SGD(lr=0.001, momentum=0.9)` and compare the accuracy/loss curves against the Adam run above; SGD typically converges slower but can generalize slightly better with enough epochs.
3. **Frozen vs fine-tuned:** already compared directly in Section 8: fine-tuning gave a +0.7% validation accuracy improvement over the frozen-base model.
4. **Precision/Recall/F1:** reported per-class in the classification report (Section 9.2) and as macro averages in Section 9.1.

12 Source Code

<https://github.com/ashrithaxx/Deep-Learning-Lab/tree/main/Experiment%204>

13 Conclusion

Transfer learning with MobileNetV2 reached 88.3% test accuracy on CIFAR-10 with under 2.6M total parameters, most of them non-trainable, proving that a lightweight pretrained backbone plus a small fine-tuning step can match or beat the accuracy of much heavier from-scratch architectures like VGG16, while training in a fraction of the time. The main weakness observed was distinguishing visually similar classes (cat/dog, deer/dog, automobile/truck), which is a limitation of the 32×32 resolution more than of the architecture itself.

14 References

1. LeCun et al., *Gradient-Based Learning Applied to Document Recognition*, Proc. IEEE, 1998.
2. Krizhevsky, Sutskever, Hinton, *ImageNet Classification with Deep CNNs*, NeurIPS, 2012.
3. Simonyan, Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. He, Zhang, Ren, Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Goodfellow, Bengio, Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>